GRAU EN ENGINYERIA INFORMÀTICA

102753 Visualització Gràfica Interactiva



3.3+.

1. Introducció

Enric Martí Gòdia

Departament de Ciències de la Computació

Continguts

1. Què és OpenGL?
2. Sintaxi OpenGL
3. Pipeline OpenGL
4. Llibreries relacionades amb OpenGL
5. Versions d'OpenGL
6. Fixed Function Pipeline (FFP)
7. Core Profile (CP)
8. Depuració codi OpenGL

1. Què és OpenGL?

OpenGL (*Open Graphics Library***):** API (*Application Programming Interface*) multi-plataforma (Win, MacOS, Linux) desenvolupada i mantinguda per Khronos Group (opengl.org).

Subministra un ampli conjunt de funcions per a manipular informació gràfica i imatges.

Característiques:

- Versions avançades (OpenGL 2.x, 3.x, 4.x) permeten programació de shaders i ús de GPU's.
- Molt utilitzada en motors i aplicacions gràfiques (Unity, Blender, etc.).

1. Què és OpenGL?

Funcionalitats:

- **Primitives gràfiques:** Triangle amb 3 arestes i 3 vèrtexs. Dibuix d'objectes basats en triangles, arestes i vèrtexs.
- **Càmera:** Posició i paràmetres del punt de vista (càmera).
- **Projeccions:** Ortogràfica, perspectiva i d'altres.
- **Ocultacions:** Algorismes *back-face culling* i Z-Buffer.
- **Il·luminació:** Visualització en filferros (sense il·luminació), cares planes i suavització (algorisme de Gouraud i Phong).
- **Efectes lumínics:** Color, textura, transparència, boira, etc.
- **Altres funcionalitats:** Implementables en shaders (GLSL): Lighting, specular i normal maps, shadow mapping, etc.

1. Què és OpenGL?

OpenGL funciona com una màquina d'estats, on cada estat és controlat per variables i objectes modificats per comandes.

Algunes variables d'estat i objectes (llistades en *red book OpenGL 3.3*+):

- Textures
- Modes de visualització
- Buffers
- Ocultacions

Comandes per a obtenir valors de les variables d'estat, segons el tipus: `glGetBooleanv()`, `glGetIntegerv()`,`glGetFloatv()`, `glGetDoublev()`.

1. Què és OpenGL?

Tipus de comandes:

- **D'ESTAT:** Defineixen propietats (tipus d'ocultacions, modus de visualització, etc.) que s'activen amb la comanda i es mantenen en el moment de crear l'escena mentre no es canviïn (`glEnable()`, `glDisable()`).
- **D'ACCIÓ:** Realitzen una acció concreta (dibuix de l'escena).

2. Sintaxi OpenGL

Exemple de codi OpenGL 1.1

Pintar rectangle blanc sobre fons negre:

```
main ()
{       Inicialitzar_Finestra();

        glClearColor(0.0,0.0,0.0,0.0);
        glClear(GL_COLOR_BUFFER_BIT);
        glColor3f(1.0,1.0,1.0);
        glOrtho(0.0,1.0,0.0,1.0,-1.0,1.0);
        glBegin(GL_POLYGON);
          glVertex3f(0.25,0.25,0.0);
          glVertex3f(0.75,0.25,0.0);
          glVertex3f(0.75,0.75,0.0);
          glVertex3f(0.25,0.75,0.0);
        gEnd();
        glFlush(); | swapBuffers();

        Actualitzar_Finestra_i_Testeja_Events();
}
```

2. Sintaxi OpenGL

Exemple de codi OpenGL 1.1 (2). Comandes:

`Inicialitzar_Finestra()`: Definició del viewport de pantalla.

`glClearColor()`: Comanda d'estat que estableix el color de fons. Negre: `(r,g,b)=(0.0,0.0,0.0)`.

`glClear()`: Comanda d'acció que esborra la pantalla.

`glColor3f()`: Comanda d'estat que estableix el color de l'objecte a dibuixar. Blanc: `(r,g,b)=(1.0,1.0,1.0)`.

`glOrtho()`: Comanda d'estat que especifica la projecció ortogràfica i el Volum de Visualització (`minX,maxX,minY,maxY,near,far`).

`glBegin()-glEnd()`: Comanda d'acció que especifica la geometria de l'objecte a dibuixar.

`glVertex3f()`:Comanda d'acció. Defineix un vèrtex de l'objecte `(x,y,z)`.

`Actualitza_Finestra_i_Testeja_Events()`: Funció que captura events de l'aplicació (canvi finestra, teclat, mouse, timer, etc.) i els tracta.

2. Sintaxi OpenGL

Sintaxi de les comandes:

- **Prefix** ‘`gl`’: Comandes de la llibreria. Exemple: `glClearColor()`.
- **Prefix** ‘`GL_`’: Constants de la llibreria. Exemple: `GL_COLOR_BUFFER_BIT`.
- **Sufixes OpenGL:**
  - Defineixen nombre i tipus dels paràmetres de la comanda.
  - Una mateixa comanda pot tenir diferents sufixes.

2. Sintaxi OpenGL

Sufixes OpenGL:

Sufix	Tipus	Tipus C	Tipus OpenGL
b	integer 8 bits	`signed char`	`GLbyte`
s	integer 16 bits	`short`	`GLshort`
i	integer 32 bits	`int`	`GLint, GLsizei`
f	flotant 32 bits	`float`	`GLfloat, GLclampf`
d	flotant 64 bits	`double`	`GLdouble,GLclampd`
ub	integer 8 bits sense signe	`unsigned char`	`GLubyte, GLboolean`
us	integer 16 bits sense signe	`unsigned short`	`GLushort`
ui	Integer 32 bits sense signe	`unsigned int`	`GLuint, GLenum`
v	vector	`pointer`	

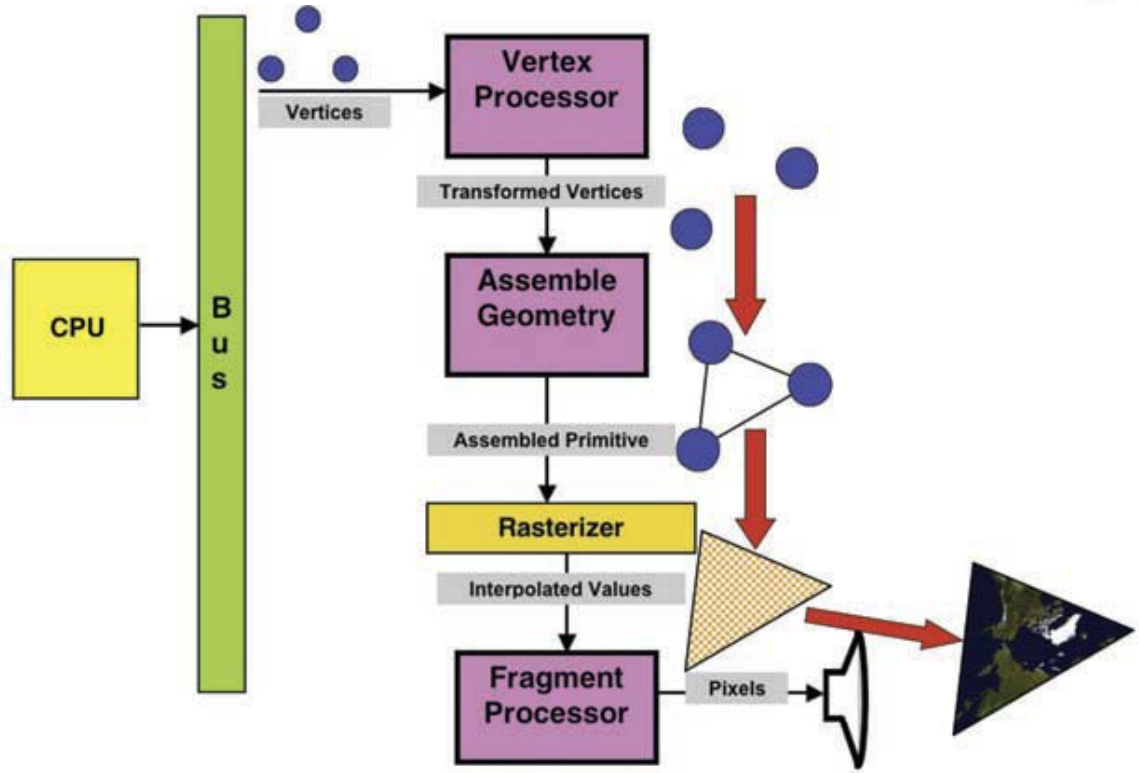
2. Sintaxi OpenGL

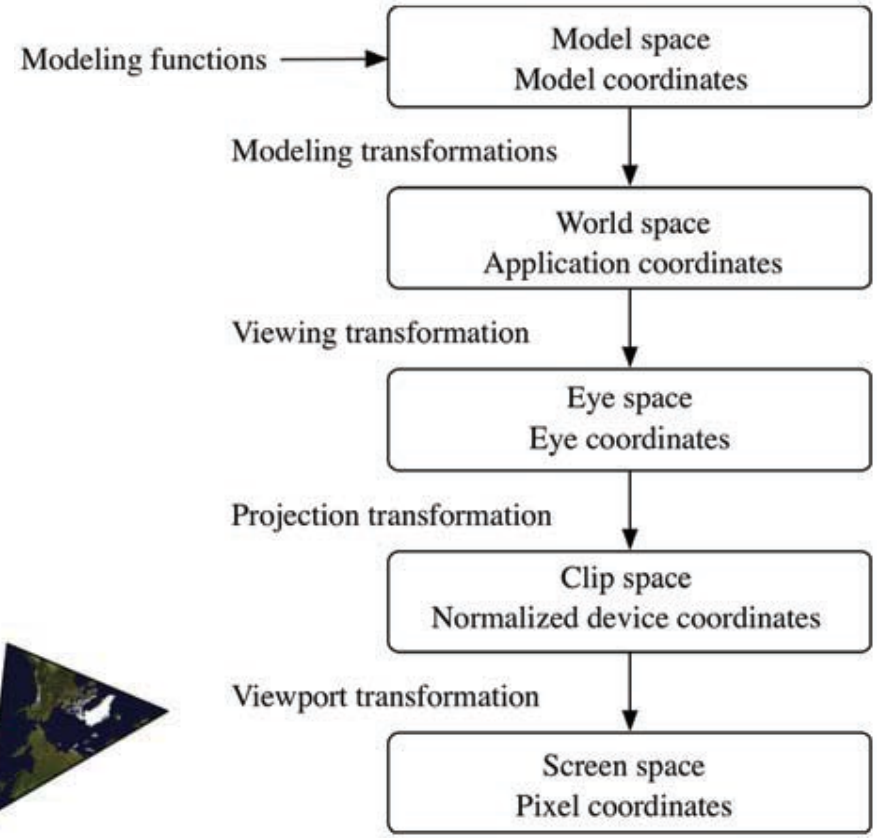
Exemple sufixes comanda OpenGL 1.1:

- `glVertex2i(1,3);` → (x,y)=(1,3)
- `glVertex2f(1.0,3.0);`
- `glVertex3i(1,3,5);` → (x,y,z)=(1,3,5)
- `glVertex3i(1.0,3.0,5.0);`
- `glVertex4i(1,3,5,1);` → (x,y,z,w)=(1,3,5,1)
- `glColor3f(0.4,0.6,0.8);` → (r,g,b)=(0.4,0.6,0.8,)
- `glColor4f(0.4,0.6,0.8,1.0);` → (r,g,b,a)=(0.4,0.6,0.8,1.0)
-
```
GLfloat Color_Array[]={1.0,0.5,1.0}
 glColor3fv(Color_Array);
```

3. Pipeline OpenGL

Seqüència de transformacions que s'apliquen a la geometria d'un objecte per a dibuixar-lo en pantalla.





3. Pipeline OpenGL

Accions per a generar una imatge:

1. Definició de paràmetres de visualització (viewport, tipus de projecció, volum de visualització, càmera).
2. Definició de transformacions geomètriques (translació, rotació i escalat) per a situar objectes en l'escena.
3. Definició de les propietats de llum: fonts de llum, tipus d'il·luminació, etc.
4. Definició de característiques lumíniques dels objectes a dibuixar (color, textura, transparències, etc.).
5. Dibuix de la geometria dels objectes (cares, arestes i vèrtexs).
6. Conversió de la geometria a píxels de pantalla (rasterització).

3. Pipeline OpenGL

Dos modes (profiles) de programar el pipeline OpenGL, lligat a les seves versions:

1. ***Fixed Function Pipeline*** (OpenGL v.1.1 fins v.3.2 en *Compatibility Mode*)
   - Majoria de funcionalitats del pipeline desenvolupades, amb poca llibertat per a programar.
   - Fàcil d'aprendre, però ineficient
2. ***Core profile*** (OpenGL 3.3+)
   - Especificació OpenGL que elimina tota la funcionalitat *fixed function pipeline* deprecada, si s'utilitza genera error.
   - Obliga a utilitzar funcionalitats modernes plaques gràfiques.
   - Molt flexible i eficient, però una mica més complex d'aprendre.
   - Conceptes: Profiles, extensions, contexts, objectes

4. Llibreries relacionades amb OpenGL

OpenGL bàsic proporciona un potent però primitiu conjunt de comandes gràfiques.

Existeixen un conjunt de llibreries de més alt nivell.

Llibreries més importants:

- GL (*OpenGL Basic Library*)
- GLU (*OpenGL Utility Library*)
- GLUT (*OpenGL Utility Toolkit*)
- GLEW (*OpenGL Extensions*)

4. Llibreries relacionades amb OpenGL

GLEW – *OpenGL Extension Wrangler Library*:

- Llibreria de càrrega d'extensions OpenGL.
- Incorpora mecanismes en temps d'execució per a determinar les extensions d'OpenGL que són suportades per la placa gràfica.
- El core d'OpenGL i les extensions estan definides en un únic fitxer de capçalera '`glew.h`'
- Codi obert i multiplataforma.
- Versió actual de glew és la 2.1.0 que inclou OpenGL 4.6.

5. Versions d'OpenGL

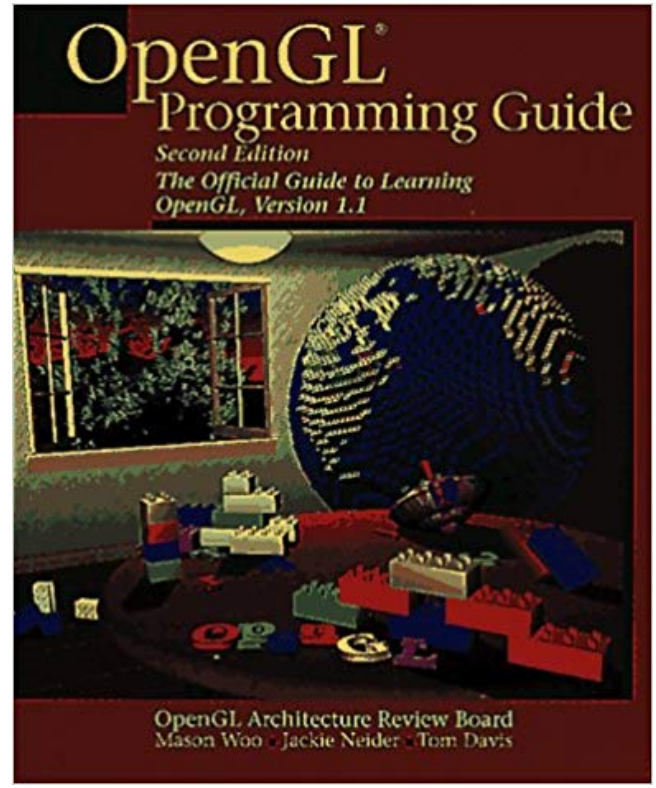
Diferents versions que augmenten les funcionalitats de la llibreria i fan evolucionar els shaders així com el llenguatge GLSL (*OpenGL Shading Language*):

1. OpenGL 1.x
2. OpenGL 2.0 / GLSL 1.10
3. OpenGL 3.x / GLSL 3.30
4. OpenGL 4.0 / GLSL 4.00
5. OpenGL ES
6. OpenGL 4.x / GLSL 4.x0

5.1. OpenGL 1.x

Versió software inicial de la llibreria que crea les bases del funcionament de la màquina d'estats i la sintaxi de les comandes.

Càlculs realitzats en software en la CPU principal (*fixed-function pipeline*).



4. Llibreries relacionades amb OpenGL

Llibreries relacionades amb OpenGL 1.1:

GL – *OpenGL Basic Library*:

- Llibreria bàsica
- Comandes amb prefix '`gl`'
- Fitxers `gl.h, opengl32.lib, opengl32.dll`

GLU – *OpenGL Utility Library*:

- Dibuix de primitives quàdriques (esfera, cilindre, disc)
- Rutines de definició de càmera - `gluLookAt()`
- Comandes amb prefix '`glu`'
- Fitxers `glu.h, glu32.lib, glu32.dll`

4. Llibreries relacionades amb OpenGL

Llibreries relacionades amb OpenGL 1.1:

GLUT – *OpenGL Utility Toolkit*:

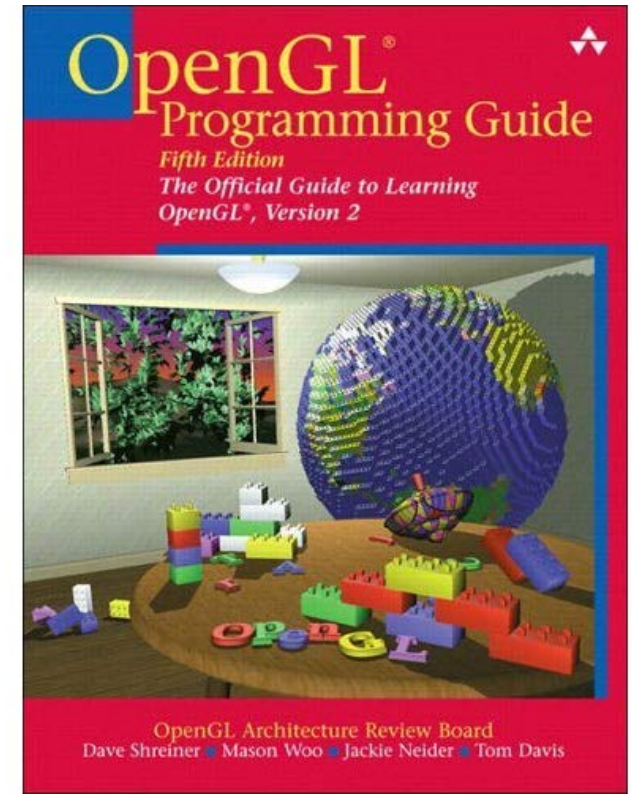
- Rutines de definició de menús i events, per a implementar aplicacions independents de plataforma.
- Dibuix de primitives més complexes (cub, piràmide, torus, con, teapot, etc.).
- Comandes amb prefix '`glut`'
- Fitxers `glut.h, glut32.lib, glut32.dll`

5.2. OpenGL 2.0 / GLSL 1.10

Primera versió que introdueix els shaders en la llibreria amb el llenguatge GLSL. La resta de versions són evolucions d'aquesta versió.

Característiques importants:

- *Vertex Shader* i *Fragment Shader*
- *Vertex Buffer Objects* (VBO) per a displaiar la geometria dels objectes
- Variables pre-definides (*built-in attribute and uniform variables*) en shader provinents de la màquina d'estats OpenGL

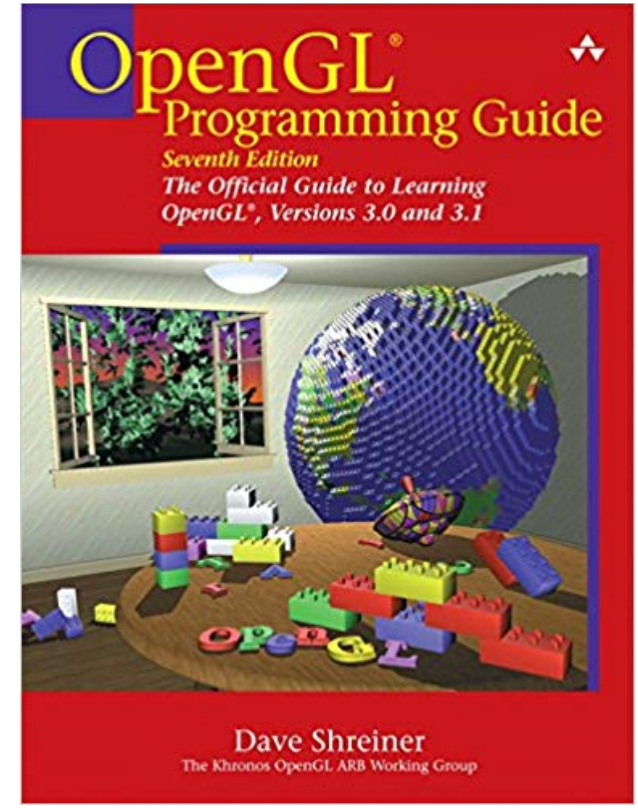


5.3. OpenGL 3.x / GLSL 3.30

Versió més revisada respecte les anteriors.

Característiques importants:

- *Geometry Shader*
- A partir v.3.3 es defineix nou perfil de funcionament del pipeline (*core profile*) que és vàlid per a totes les versions porteriors
- Manté compatibilitat amb funcions de *fixed-function pipeline* marca com a *deprecated, principalment*:
  - Ús de `glBegin()-glEnd()` per a displaiar geometria. Només permet l'ús de *Vertex Arrays* (VA) i/o *Vertex Buffer Objects* (VBO)
  - Sense variables pre-definides (*built-in attribute and uniform variables*) provinents de la màquina d'estats OpenGL



5.4. OpenGL 4.0 / GLSL 4.00

Darrera versió important.

Característiques importants:

- *Tesselation Shader*
- Desenvolupaments importants en el **Geometry Shader** (crear múltiples instancies del shader).
- Permet implementar una pròpia interpolació de textures sobre un fragment a partir dels quatre veïns.
- GLSL inclou comandes `#include` i tipus `double`.

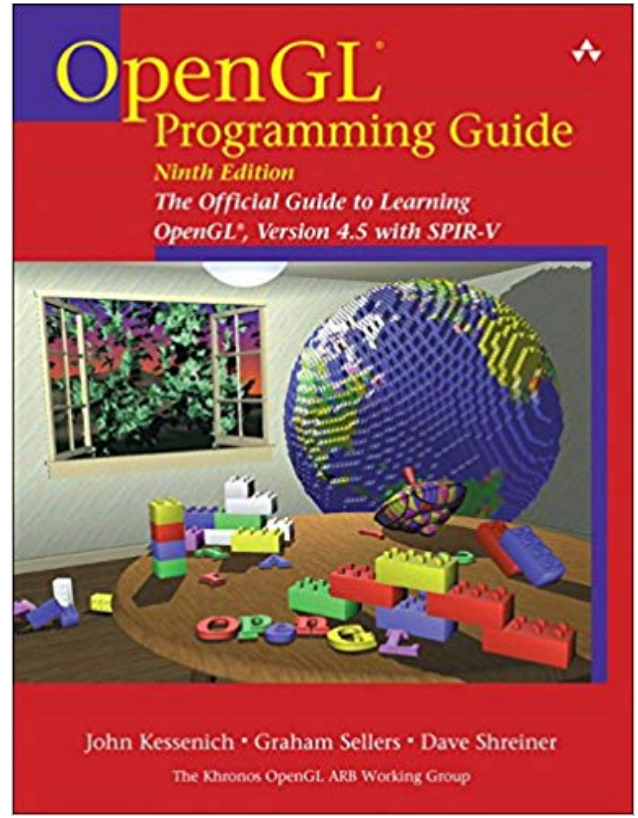
5.5. OpenGL ES

ES - *Embedded Systems*.

Llibreria per a desenvolupar aplicacions gràfiques en mòbils.

Característiques importants:

- Tot el processament es fa en els shaders (no *fixed-function operations*).
- Només *Vertex Shader* i *Fragment Shader.*
- Sense variables pre-definides (*built-in attribute and uniform variables*) provinents de la màquina d'estats OpenGL.
- Codi de shader en *OpenGL ES shader language 1.10* s'executa en OpenGL 4.1.



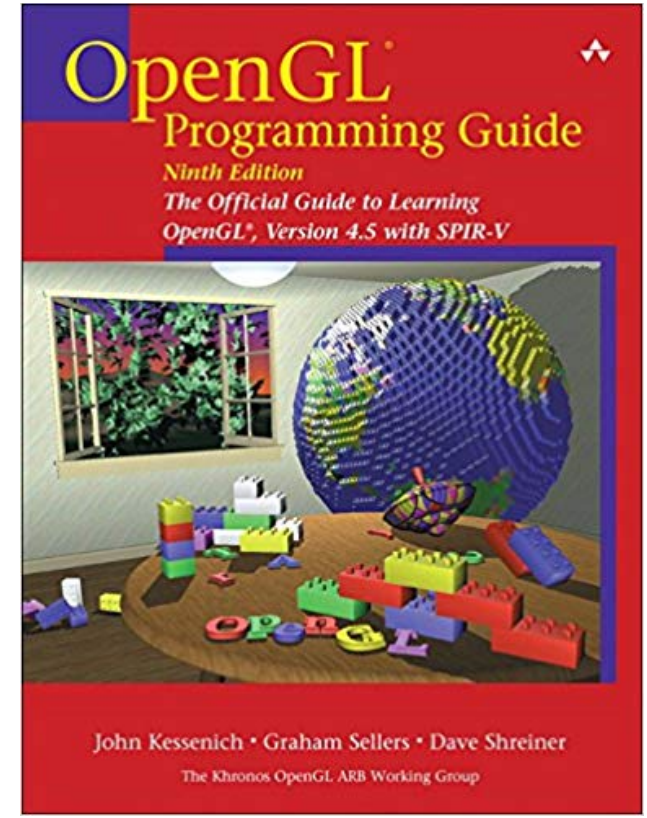
5.6. OpenGL 4.x

Família de versions més recents. La darrera és la 4.6.

No totalment extesa en plaques.

Característiques importants:

- *Shader binaries*: poder compilar shaders i guardar-los en fitxer, per a carregar-los posteriorment.
- Shaders poden modificar (llegir o escriure) imatges textura.
- Compatibilitat amb OpenGL ES 2.0. Codi OpenGL ES pot ser executat en OpenGL 4.x



6. Fixed Function Pipeline (FFP)

També anomenat *immediate mode* o *legacy OpenGL*. Components més importants deprecats:

- Especificació d'atributs per a vèrtexs:

```
glBegin(GL_POLYGON);
  glVertex3f(0.25,0.25,0.0);
  glVertex3f(0.75,0.25,0.0);
  glVertex3f(0.75,0.75,0.0);
gEnd();
```

- Modes `GL_QUAD` i `GL_POLYGON`
- Estructura *Display Lists*
- Stack de matrius i comandes per a TG, projeccions i càmeres: `glMatrixMode()`, `glTranslatef()`, `glRotatef()`, `glOrtho()`, `gluPerspective()`, `gluLootAt()`
- Definició llums, il·luminació i materials: `glLightfv()`, `glLightModel()`, `glEnable(GL_LIGHT0)`, `glMaterial()`,

6. Fixed Function Pipeline (FFP)

En què afecta als programadors aquesta supressió:

- Major control del pipeline de visualització mitjançant la programació amb shaders d'aquestes funcionalitats amb el llenguatge GLSL (*GL Shading Language*).
- Ús de comandes avançades i optimitzades en hardware per a visualització de geometria (VBO, VAO, EBO).
- Implementació pròpia d'estructures de matrius i funcions per a TG, projecció i càmera.
- Implementació pròpia d'efectes visuals (il·luminació, materials, ombres, etc.) o utilitzant algorítmica implementada en plaques gràfiques.
- Ús de llibreries externes (GLFW, GLM, TVMET, CGLM, Assimp, LIB3DS, ASSETKIT, etc.)

6. Fixed Function Pipeline (FFP)

Raons per no utilitzar FFP:

- **Eficiència:** Geometria enviada directament a GPU, simplificant crides i pas per CPU.
- **Flexibilitat per l'ús dels shaders:** Shaders enlloc processament FFP. Major control i responsabilitat.
- **Algorismes d'il·luminació més complexes:** FFP només implementa algorismes *flat* i Gouraud. Phong del més implementat però no en FFP.
- ***Shadow mapping*:** Programable amb shaders. GLSL amb funcions d'ombres.
- **Il·luminació no foto-realística:** *Toon shading*, *cell shading*.
- **Animació:** Animar amb textures o TG sense l'aplicació.

7. Core Profile (CP)

Funcionament d'OpenGL a partir de la v.3.2, que trenca radicalment amb les versions anteriors:

- Manteniment de funcions molt bàsiques com esborrat pantalla, definició de textures, visualització de punts, filferros, entre altres.
- Definició de contextes, perfils (*profiles*), extensions i objectes que permeten utilitzar les noves funcionalitats incorporades en el hardware gràfic pels fabricants.
- **Khronos Group** defineix especificacions de les versions, extensions i funcions a implementar pels fabricants.
- A la vegada, fabricants poden definir extensions pròpies, que en posteriors versions s'adoptin com estàndard.
- Sintaxi estàndard de prefixes i sufixes OpenGL.

7.1. Contexte

Màquina d'estats que magatzema tota la informació d'una instància OpenGL.

- Inclou: variables d'estat, comandes, buffers, etc.
- Definit dins l'execució d'una aplicació i sobre un SO.
- Es poden crear múltiples contextes, un per a cada finestra de l'aplicació.
- Tota comanda OpenGL s'aplica sobre un context actiu (*current context*).
- Cada thread assignat a un context OpenGL, un sol contexte no pot definir múltiples *threads* simultàniament.

7.1. Contexte

En OpenGL, diferents contexts:

- Perfils (profiles)
- **DEBUG OUTPUT**: Context que facilita debuggar i optimitzar OpenGL (OpenGL v.4.3+).

Cada context conté:

- Extensions
- Objectes

7.2. Perfils (profiles)

Modes de funcionament, que agrupen funcionalitats, dins una màquina d'estats (context).

Fins a v.3.2 especificacions sense perfils.

A partir v.3.3, dos perfils que poden conviure en el context:

- ***Core profile*:** Funcionament basat en shaders
- ***Compatibility Profile*:** Funcionament FFP, per mantenir funcionament aplicacions basades en FFP.

Tota implementació de fabricant a partir de la v.3.3 ha de tenir *Core Profile*, no obligat *Compatibility Profile*, malgrat la majoria la incorpora.

Perfils tractats com extensions:

- `ARB_CREATE_CONTEXT_PROFILE`**:** per a *Core Profile.*
- `ARB_COMPATIBILITY`: per a *Compatibility Profile*.

7.2. Perfils (profiles)

FORWARD COMPATIBILITY BIT SET (FCBS): Si s'activa per a un perfil donat, fa que totes les funcionalitats *deprecated* siguin eliminades.

Necessari per a crear un contexte amb core profile.

A partir v.3.3 (OpenGL 3.3+), FCBS segons perfils:

- ***Core profile*:** Tota funcionalitat *deprecated* és el.liminada
- ***Compatibility Profile:*** No afecta en res, doncs en aquest perfil no hi ha funcionalitat *deprecated*

7.3. Extensions

Nova tècnica o optimització de visualització implementada i desenvolupada per un fabricant.

Si el hardware suporta l'extensió, el programador pot utilitzar-la en l'aplicació.

Forma d'expandir funcionalitats en el core OpenGL, doncs el fabricant pot incorporar en la seva placa nous algorismes sense esperar que siguin acceptats com estàndard en OpenGL.

Pràctica acceptada per Khronos Group.

Sovint, aquestes noves extensions s'incorporen a futures versions OpenGL com estàndard.

7.3. Extensions

Tipus d'extensions:

- **Estàndard:** Acceptades per Khronos, pròpies de cada versió OpenGL (`GL_ARB_*`). ARB: *Architecture Review Board*.
- **Owner:** Pròpies d'un fabricant o SO:
  - `GL_SGI_*`, `GL_SGIX`: Silicon Graphics
  - `GL_NV_*`, `GL_NVX_*` : NVIDIA
  - `GL_ATI_*` : ATI
  - `WGL_ARB_*`, `WGL_NV_*`, `WGL_ATI_*` : Windows
  - `AGL_ARB_*`, `AGL_NV_*`, `AGL_ATI_*` : Apple

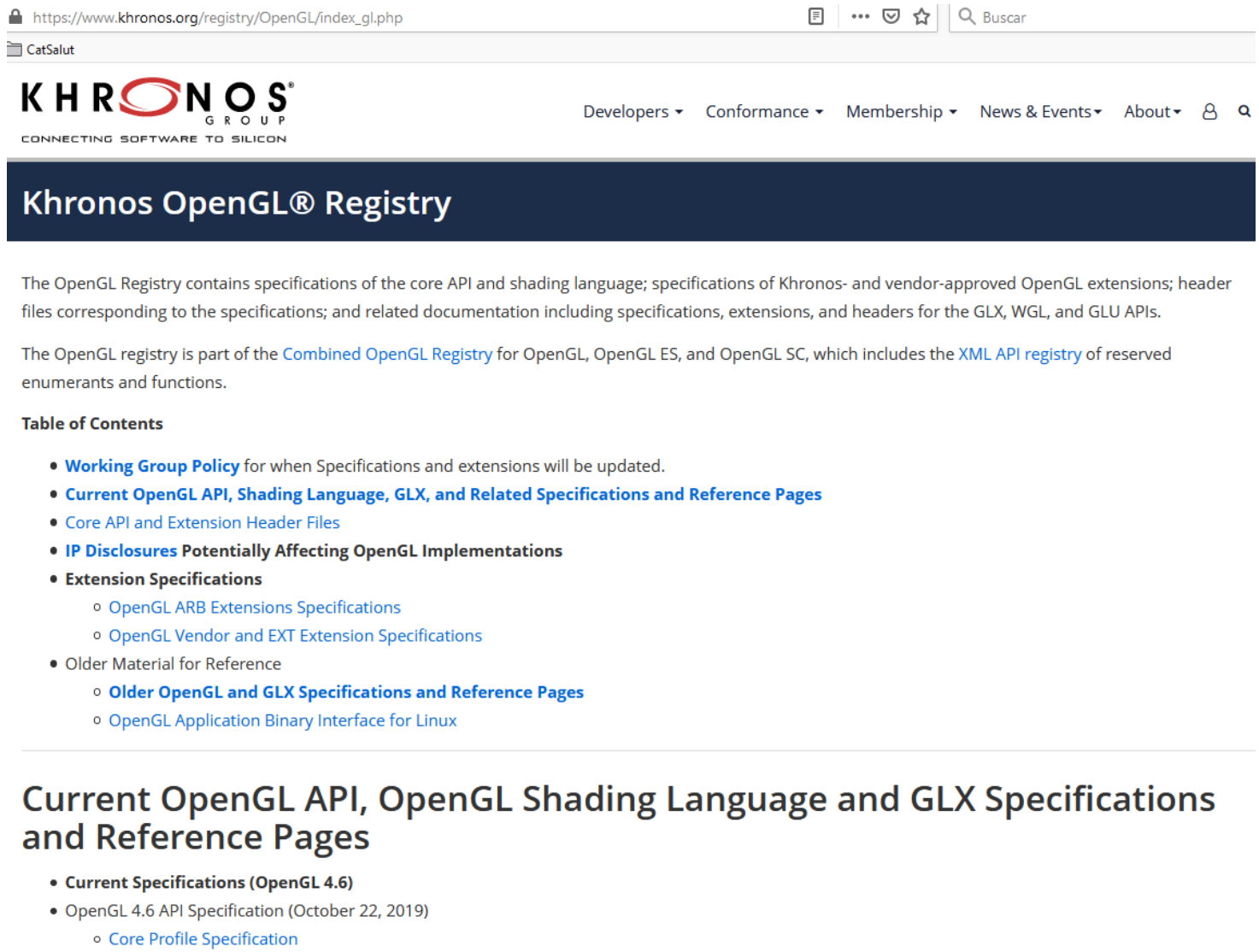
7.3. Extensions

OpenGL Extensions Viewer: Software lliure que permet inspeccionar la versió, cores i extensions que té la nostra placa gràfica. Disponible en Windows, MacOS, Android, Iphone, Ipad, etc.

OpenGL Extension Registry:

- Registre de totes les especificacions, versions i extensions OpenGL acceptades per a l'estàndard (unes 300) i especificació del llenguatge GLSL per a shaders.
- Mantingut per *Architectural Review Board* (ARB), grup responsable de l'API OpenGL.
- https://www.khronos.org/registry/OpenGL/index_gl.php

7.3. Extensions



7.3. Extensions

Continguts d'una extensió:

- Nom de l'extensió - `GL_ARB_*`
- Tokens (constants) - `GL_*_ARB`
- Funcions – `gl*_ARB()`

Exemple:

Extensió	Token	Funcions
`GL_ARB_MULTITEXTURE`	`GL_TEXTURE0_ARB`	`glActiveTextureARB()`
`GL_ARB_sparse_buffer`	`SPARSE_STORAGE_BIT_ARB`	`glBufferPageCommitmentARB()`

Totes les extensions s'apliquen sobre una versió d'OpenGL com un afegit del seu core.

7.3. Extensions

Test d'una extensió: Programador pot testejar dins l'aplicació si el hardware gràfic soporta l'extensió per a utilitzar-la, o definir l'alternativa si no la té.

```
const char* GLExtensions = glGetString(GL_EXTENSIONS);
if isExtensionSupported(GL_ARB_extension_name, GLExtensions)
      { <executa funció de l'extensió> }
      else { <executar algorisme alternatiu>}
```

També es pot testejar cada extensió per separat per l'index k amb les funcions:

```
glGetIntegerv(GL_NUM_EXTENSIONS,Num_Ext);
glGetString(GL_EXTENSIONS,k); // amb (k=0; k<Num_Ext;i++)
```

7.3. Extensions

```
// Helper to check for extension string presence. Adapted from:
// http://www.opengl.org/resources/features/OGLextensions/
static int isExtensionSupported(const char *extList, const char
*extension) {
const char *start;
const char *where, *terminator;
// Extension names should not have spaces.
where = strchr(extension, ' ‘);
if ( where || *extension == '\0' ) return 0;
/* It takes a bit of care to be fool-proof about parsing the OpenGL
extensions string. Don't be fooled by sub-strings, etc. */
for ( start = extList; ; ) {
    where = strstr( start, extension );
    if ( !where ) break;
    terminator = where + strlen( extension );
    if ( where == start || *(where - 1) == ' ‘ )
        if ( *terminator == ' ' || *terminator == '\0' ) return 1;
    start = terminator; }
return 0; }
```

7.3. Extensions

Càrrega d'extensions: Es carreguen en temps d'execució.

Dues formes:

- **Automàtica:** Mitjançant una llibreria com GLEW, GLEE, etc.
- **Manual:** Mitjançant funcions.

7. Core Profile (CP)

Obtenció d'informació del core i contexte:

- ***OpenGL version number:*** Digit major i menor versió que soporta la implementació OpenGL.

```
glGetIntegerv(GL_MAJOR_VERSION,*);
glGerInteger(GL_MINOR_VERSION,*);
```

- **Vendor:** Fabricant de la implementació (NVIDIA, ATI, etc.)

```
glGetString(GL_VENDOR);
```

- **Renderer Name:** Nom del renderer

```
glGetString(GL_VENDOR);
```

- **Extensions List:** Llistat d'extensions

```
glGetIntegerv(GL_NUM_EXTENSIONS,*);
glGetString(GL_EXTENSIONS);
glGetString(GL_EXTENSIONS,k);
```

7. Core Profile (CP)

```
char* str = (char*)glGetString(GL_VENDOR);
FILE* f = fopen(nomfitxer, "w");
fprintf(stderr, "VENDOR: %s\n", str);

str = (char*)glGetString(GL_RENDERER);
fprintf(stderr, "RENDERER: %s\n", str);

str = (char*)glGetString(GL_VERSION);
fprintf(stderr, "VERSION: %s\n", str);

str = (char*)glGetString(GL_SHADING_LANGUAGE_VERSION);
fprintf(stderr, "SHADING_LANGUAGE_VERSION: %s\n", str);

str = (char*)glGetString(GL_EXTENSIONS);
fprintf(stderr, "EXTENSIONS: %s\n", str);
```

7.4. Objectes

Estructura que representa un subset del context (màquina d'estats) d'OpenGL, semblant a una classe en programació OO, amb variables i comandes.

Exemple: Definició d'una finestra de visualització on es defineixen el tamany, color de fons, etc.

Variable: Representable com una estructura C:

```
struct object_name
{
        GLfloat option 1;
        GLfloat option 2;
        Glchar[ ] name;
};
```

7.4. Objectes

Segons això, el context OpenGL pot ser vist com un conjunt d'objectes instanciats:

```
// Estat OpenGL
struct OpenGL_Context ´
       ...
       object* object_window_target;
       ...
}
```

Important: Podem definir vàries instàncies del mateix objecte, configurar cadascuna i quan es vulgui utilitzar en vàries còpies s'activa (*bind*) en el context i es crida.

7.4. Objectes

Comandes per a configurar l'objecte: Un sol cop

```
// Crear objecte
GLuint objectId=0;
glGenObject(1,&objectID);

// Bind the object in the context
glBindObject(GLWINDOW_TARGET, objectId);

// Set options of the object based in GL_WINDOW_TARGET
glSetObjectOption(GL_WINDOW_TARGET,GL_OPTION_WINDOW_WIDTH,800);
glSetObjectOption(GL_WINDOW_TARGET,GL_OPTION_WINDOW_HEIGHT,600);

// Set context target back to default (unbind the object)
glBindObject(GL_WINDOW_TARGET,0);
```

7.4. Objectes

Comandes per a cridar l'objecte: Cada cop que se'n vulgui fer una còpia.

```
// Bind the object in the context
glBindObject(GLWINDOW_TARGET, objectId);

// Call the object
glCallObject(GL_WINDOW_TARGET);

// Set context target back to default (unbind the object)
glBindObject(GL_WINDOW_TARGET,0);
```

7.4. Objectes

Dos categories:

Objectes regulars:

- *Vertex Buffer Object* (VBO)
- *Element Buffer Object* (EBO)
- *Texture Object*
- *Renderbuffer Object*, etc.

Objectes contenidor:

- *Vertex Array Object* (VAO)
- *Framebuffer Object*, etc.

8. Depuració codi OpenGL

OpenGL proporciona eines per a poder testejar el codi OpenGL:

1. Gestió d'Errors
2. Extensió `ARB_DEBUG_OUTPUT`

8.1. Gestió d'Errors

Si s'utilitza una comanda OpenGL de forma incorrecte, es genera uns flags d'error.

Podem consultar aquests flags amb la funció

```
GLenum glGetError();
```

que llista un codi d'error:

Flag	Code	Description
GL_NO_ERROR	0	No user error reported since last call to glGetError.
GL_INVALID_ENUM	1280	Set when an enumeration parameter is not legal.
GL_INVALID_VALUE	1281	Set when a value parameter is not legal.
GL_INVALID_OPERATION	1282	Set when the state for a command is not legal for its given parameters.
GL_STACK_OVERFLOW	1283	Set when a stack pushing operation causes a stack overflow.
GL_STACK_UNDERFLOW	1284	Set when a stack popping operation occurs while the stack is at its lowest point.
GL_OUT_OF_MEMORY	1285	Set when a memory allocation operation cannot allocate (enough) memory.
GL_INVALID_FRAMEBUFFER_OPERATION	1286	Set when reading or writing to a framebuffer that is not complete.

8.1. Gestió d'Errors

La funció `glGetError()` també resseteja els flags d'error, un cop llista l'error:

```
glBindTexture(GL_TEXTURE_2D, tex);
std::cout << glGetError() << std::endl; // Returns 0 (no error)

glTexImage2D(GL_TEXTURE_3D,0,GL_RGB,512,512,0,GL_RGB,GL_UNSIGNED_BYTE,data);
std::cout << glGetError() << std::endl; // Returns 1280 (invàlid enum)

glGenTextures(-5,textures);
std::cout << glGetError() << std::endl; // Returns 1281 (invàlid value)

std::cout << glGetError() << std::endl; // Returns 0 (no error)
```

Per a no memoritzar codis d'error, cal definir la funció `glCheckError()` que mostra missatges d'error.

8.1. Gestió d'Errors

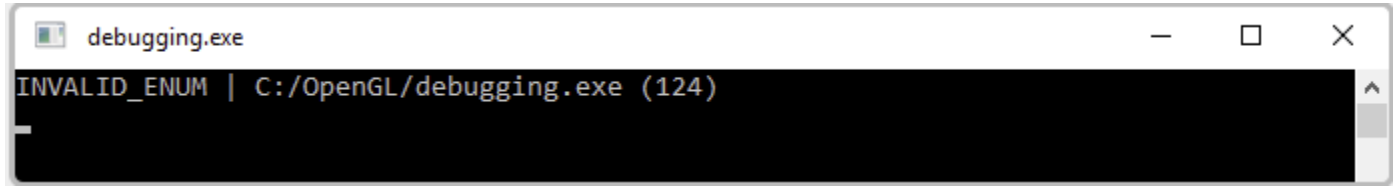
```
GLenum glCheckError_(const char *file, int line)
{   GLenum errorCode;
    while ((errorCode = glGetError()) != GL_NO_ERROR)
    {    std::string error;
         switch (errorCode)
         { case  GL_INVALID_ENUM:      error = "INVALID_ENUM";     break;
           case  GL_INVALID_VALUE:     error = "INVALID_VALUE";    break;
           case  GL_INVALID_OPERATION: error = "INVALID_OPERATION"; break;
           case  GL_STACK_OVERFLOW:    error = "STACK_OVERFLOW";   break;
           case  GL_STACK_UNDERFLOW:   error = "STACK_UNDERFLOW";  break;
           case  GL_OUT_OF_MEMORY:     error = "OUT_OF_MEMORY";    break;
           case GL_INVALID_FRAMEBUFFER_OPERATION:
                                error="INVALID_FRAMEBUFFER_OPERATION"; break;
          }
    std::cout << error << " | " << file << " (" << line << ")" << std::endl;
    }
return errorCode;
}
#define glCheckError() glCheckError_(__FILE__, __LINE__)
```

8.1. Gestió d'Errors

Ús `glCheckError()`:

```
glBindTexture(GL_TEXTURE_2D, tex);
glCheckError();
```

Resposta:



8.2. Extensió ARB_DEBUG_OUTPUT

Extensió OpenGL a partir de la versió 4.3.

Per a no posar la funció de test d'error en cada comanda OpenGL, aquesta extensió permet definir una funció de *callback*, de forma que quan es produeix un error es crida aquesta funció que reportarà l'error.

L'activació d'aquesta extensió pot alentir l'execució de l'aplicació. Per a la versió de producció (release) és preferible desactivar aquesta extensió.

8.2. Extensió ARB_DEBUG_OUTPUT

Passos:

1. Activació de l'extensió (en GLFW):

```
glfwWindowHint(GLFW_OPENGL_DEBUG_CONTEXT, GL_TRUE);
```

2. Configurar l'extensió, definint `glDebugOutput()` com a funció de callback:

```
GLint flags; glGetIntegerv(GL_CONTEXT_FLAGS, &flags);
if (flags & GL_CONTEXT_FLAG_DEBUG_BIT)
{  glEnable(GL_DEBUG_OUTPUT);
   glEnable(GL_DEBUG_OUTPUT_SYNCHRONOUS);
   glDebugMessageCallback(glDebugOutput,nullptr);
   glDebugMessageControl(GL_DONT_CARE,GL_DONT_CARE, GL_DONT_CARE, 0,
                         nullptr, GL_TRUE);
}
```

8.2. Extensió ARB_DEBUG_OUTPUT

Amb `glDebugMessageControl()` podem filtrar els tipus d'errors que volem reportar.

Amb `GL_DONT_CARE` especifiquem que no volem filtrar cap error, que es reportin tots quan es produeixin.

Exemple: Si només volem mostrar errors de l'API OpenGL que tinguin severitat alta:

```
glDebugMessageControl(GL_DONT_CARE,GL_DONT_CARE,GL_DONT_CARE,
                      0,nullptr,GL_TRUE);
```

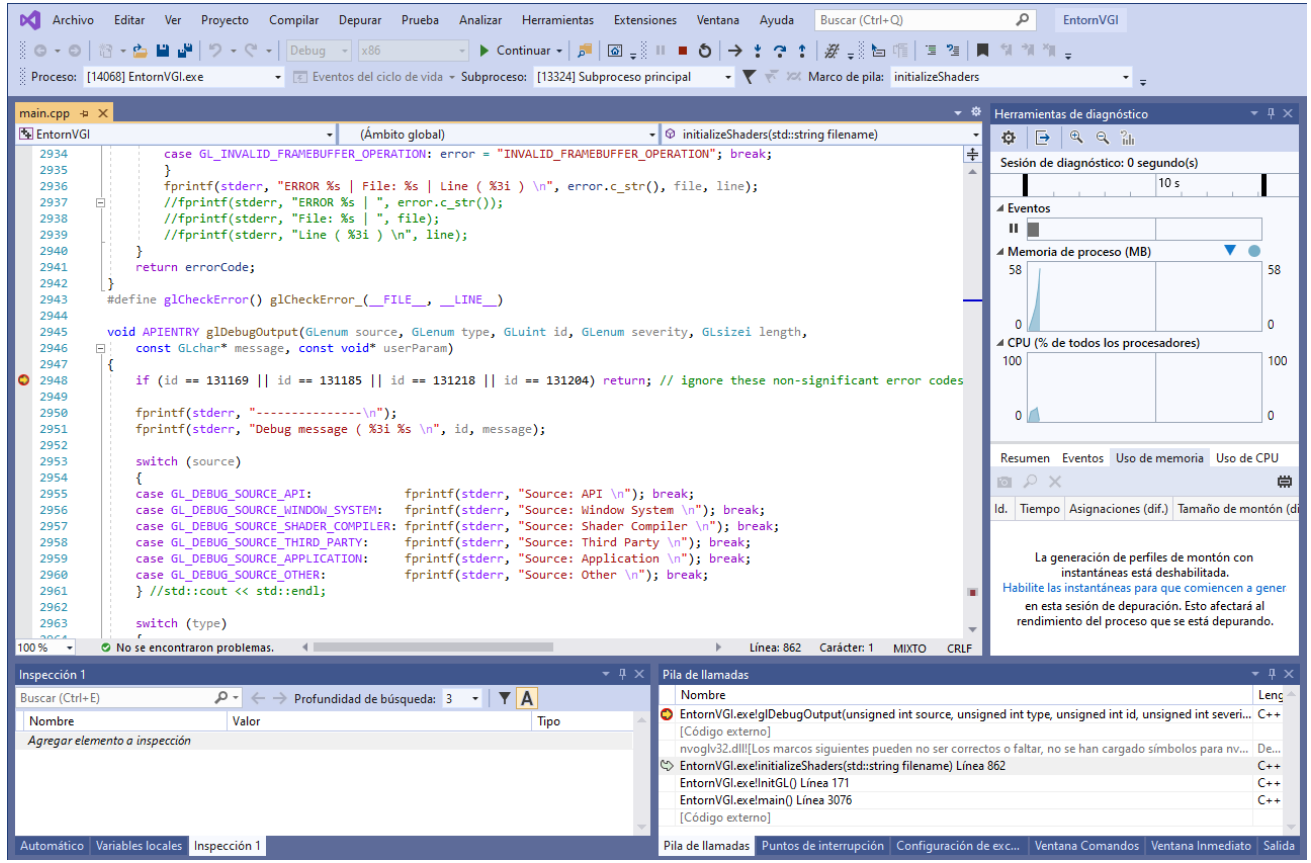
8.2. Extensió ARB_DEBUG_OUTPUT

Funció `glDebugOutput()` dóna bastant informació sobre l'error que es produeix:

```
void APIENTRY glDebugOutput(GLenum source, GLenum type, GLuint id, GLenum severity, GLsizei length, const
GLchar *message, void *userParam)
{if(id==131169 || id==131185 || id==131218 || id==131204) return; //Ignore non-significant error/warning codes
std::cout << "---------------" << std::endl; std::cout<<"Debug message (" << id << "): "<<message<< std::endl;
switch (source)
{ case GL_DEBUG_SOURCE_API:                     std::cout << "Source: API"; break;
  case GL_DEBUG_SOURCE_WINDOW_SYSTEM:           std::cout << "Source: Window System"; break;
  case GL_DEBUG_SOURCE_SHADER_COMPILER:         std::cout << "Source: Shader Compiler"; break;
  case GL_DEBUG_SOURCE_THIRD_PARTY:              std::cout << "Source: Third Party"; break;
  case GL_DEBUG_SOURCE_APPLICATION:             std::cout << "Source: Application"; break;
  case GL_DEBUG_SOURCE_OTHER: std::cout << "Source: Other"; break; }    std::cout << std::endl;
switch (type)
{  case GL_DEBUG_TYPE_ERROR:                    std::cout << "Type: Error"; break;
  case GL_DEBUG_TYPE_DEPRECATED_BEHAVIOR:       std::cout << "Type: Deprecated Behaviour"; break;
  case GL_DEBUG_TYPE_UNDEFINED_BEHAVIOR:        std::cout << "Type: Undefined Behaviour"; break;
  case GL_DEBUG_TYPE_PORTABILITY:               std::cout << "Type: Portability"; break;
  case GL_DEBUG_TYPE_PERFORMANCE:               std::cout << "Type: Performance"; break;
  case GL_DEBUG_TYPE_MARKER:                    std::cout << "Type: Marker"; break;
  case GL_DEBUG_TYPE_PUSH_GROUP:                std::cout << "Type: Push Group"; break;
  case GL_DEBUG_TYPE_POP_GROUP:                 std::cout << "Type: Pop Group"; break;
  case GL_DEBUG_TYPE_OTHER:                     std::cout << "Type: Other"; break; } std::cout << std::endl;
switch (severity)
{ case GL_DEBUG_SEVERITY_HIGH:                  std::cout << "Severity: high"; break;
  case GL_DEBUG_SEVERITY_MEDIUM:                std::cout << "Severity: medium"; break;
  case GL_DEBUG_SEVERITY_LOW:                   std::cout << "Severity: low"; break;
  case GL_DEBUG_SEVERITY_NOTIFICATION: std::cout << "Severity: notification"; break; } std::cout << std::endl;
std::cout << std::endl; }
```

8.2. Extensió ARB_DEBUG_OUTPUT

Backtracking en el codi font: En mode *debugger*, posant punt de parada en la funció `glDebugOutput()`, podem saber quina comanda ha produït l'error seguint la pila de trucades.



En resum...

- L'API OpenGL és una potent llibreria gràfica multiplataforma que s'utilitza en programes de disseny i motors gràfics.
- Es basa en llenguatge C i té una sintaxi basada en prefixes 'gl' o 'GL_', i sufixes indicant el nombre i tipus de paràmetres.
- Ha sigut una llibreria que s'ha adaptat a l'evolució del hardware gràfic mitjançant versions i extensions. Actualment en la versió 4.6.
- Programar una aplicació en versions massa novedoses pot ser que no sigui compatible amb tots els ordinadors. Cal detectar si es disposa o no de l'extensió per aprofitar al màxim el hardware gràfic.